

Schema-Augmented Retrieval for Text-to-SQL: Natural Language Access to the Chicago Crime Database

Vignesh, Rupesh, Alexander, Xhonping, and Fayeq

Abstract

As data-driven decision-making becomes increasingly critical across organizations, non-technical stakeholders often cannot write SQL queries to extract insights from databases. To address this Text-to-SQL natural language processing (NLP) problem, this project proposes a SQL chatbot built with prompt engineering. The system is designed as an AI agent using LangChain or LangGraph to manage interactions. The agent translates natural language into SQL queries, executes them against the database, and summarizes the resulting data back into natural language. The system’s query generation accuracy and robustness are evaluated using the Chicago Crime dataset, a large-scale structured dataset containing records of criminal incidents with attributes such as location, time, and offense type. This project aims to expand data access by removing the requirement for SQL knowledge.

1 Introduction

Relational databases are critical for operational intelligence, but their data remain largely inaccessible to non-technical stakeholders who lack SQL proficiency. This barrier forces business users to rely on engineering teams, creating operational bottlenecks and delaying actionable insights.

Although LLMs have introduced Text-to-SQL capabilities, current systems struggle in real-world settings. Low-parameter or locally hosted models frequently hallucinate schemas, generate invalid queries, and lack intrinsic self-correction capabilities. Furthermore, many systems stop at query generation, leaving users to interpret raw tables.

This project addresses these limitations by developing an autonomous Text-to-SQL agent. By focusing on advanced prompt engineering techniques, the system uses an iterative LangGraph loop to compensate for the reasoning limitations of accessible models, enabling it to produce accurate SQL and synthesize results into conversational language reliably.

1.1 Literature Survey

Recent Text-to-SQL research heavily focuses on schema-aware encoding and in-context learning, as demonstrated in benchmark datasets such as Spider [1] and models like RAT-SQL [2]. Frameworks like DAIL-SQL [3] and DIN-SQL [4] demonstrate that structured, decomposed prompting (e.g., dynamic example selection and schema linking) significantly improves execution accuracy compared to standard zero-shot methods. However, these frameworks are predominantly based on massive proprietary LLMs. They act as static pipelines that fail when applied to resource-constrained models such as SQLCoder [5], which cannot intrinsically self-correct syntax errors.

To address static generation, multi-agent frameworks (e.g., MAC-SQL [6]) utilize iterative observation-execution loops based on the ReAct paradigm to autonomously fix SQL errors. While these improve syntax accuracy, they are computationally heavy and terminate by returning raw data tables, ignoring the end-user experience.

This project builds upon the decomposed reasoning of DIN-SQL and the iterative loops of MAC-SQL but diverges by adapting these strategies for low-parameter models using LangGraph [7]. Additionally, it introduces a final natural language synthesis step, fully closing the operational gap for non-technical users.

These design choices are particularly important in light of recent benchmarks such as BIRD [8], which highlight the challenges of real-world Text-to-SQL tasks under complex schema and reasoning requirements.

1.2 Proposed Methodology

The system is evaluated on the Chicago Crime database. It contains structured records of reported incidents, including attributes such as location, time, and offense type, making it suitable for evaluating complex Text-to-SQL queries involving filtering, aggregation, and temporal reasoning. To ensure zero-latency testing, the environment uses PostgreSQL running in Docker, with an interface layer utilizing SQLAlchemy and psycopg2 for secure, programmatic database connectivity. To maintain data recency, Apache Airflow manages a weekly Extract, Transform, and Load (ETL) pipeline that refreshes the dataset. For security, a “Read-Only” role is enforced at the database level to ensure the agent never accidentally executes destructive commands, such as DROP TABLE.

The core system is an autonomous agent orchestrated via LangGraph, which handles the stateful logic, retries, and “thought” loops. The system evaluates models across both Groq (via API calls) and Ollama (running locally). The LangGraph workflow operates as a self-correction loop:

- **Generate:** The LLM attempts the SQL, utilizing Few-Shot Learning tailored to handle dataset-specific queries.
- **Validate:** A Python node attempts to “dry run” the query.
- **Catch:** If psycopg2 throws an error, the error message is fed back to the LLM.
- **Refine:** The LLM sees the error and generates a corrected version.

This process repeats until the query is valid or the retry limit is hit. Finally, the tabular results are narrated back in plain English. The system is also designed to handle dataset ambiguity, such as mapping colloquial user terms (e.g., “crimes in the Loop”) to specific Community Area IDs.

Performance is rigorously assessed using 100 hand-crafted natural language questions paired with ground-truth SQL queries, stratified by difficulty (Easy, Medium, Hard). Evaluation metrics include: Execution Accuracy (EX): Percentage of generated queries returning an identical result set to the ground-truth SQL. Valid SQL Rate (VSR): Percentage of queries that execute without triggering a syntax error. Synthesis Quality: LLM-as-a-Judge grading on a 1-to-5 scale for the clarity and fluency of the final natural language output.

1.3 Contributions

- **A Novel End-to-End Agentic Architecture:** We implement a comprehensive SQL agent utilizing LangGraph. This architecture introduces a localized self-correction execution loop and narrates data results back in plain English, successfully closing the operational loop for non-technical end-users.
- **Rigorous Comparative Analysis of Prompting and Models:** We systematically test how different Prompt Engineering techniques influence the accuracy and reliability of open-source models accessed via Groq and locally through Ollama. This analysis quantifies the performance delta in EX and VSR between standard prompting and our iterative framework.

2 Materials and Methods

2.1 Dataset and Data Processing

2.1.1 Dataset Description

This study utilizes the **Chicago Crime Dataset**, sourced from the City of Chicago’s official open data portal (<https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2>). The dataset comprises approximately **8.5 million crime records** spanning over two decades. Each record contains 22 attributes:

- **Categorical Attributes:** CRIME_TYPE, DESCRIPTION, LOCATION_DESCRIPTION, IUCR, BEAT, WARD, DISTRICT, COMMUNITY_AREA
- **Temporal Attributes:** DATE, UPDATED_ON, YEAR

- **Geospatial Attributes:** LATITUDE, LONGITUDE, X_COORDINATE, Y_COORDINATE, LOCATION (point geometry)
- **Metadata Attributes:** ID, CASE_NUMBER, ARREST (Boolean), DOMESTIC (Boolean)

The dataset consists of structured tabular data in CSV format, which is gathered through weekly API pulls from the Chicago Data Portal orchestrated by Apache Airflow. For storage infrastructure, the project utilizes a PostgreSQL v14+ database containerized with Docker running in our remote server to ensure environmental reproducibility.

2.1.2 Data Processing Pipeline

Schema Standardization All column names were transformed from mixed case with spaces (e.g., “Primary Type”) to uppercase with underscores (e.g., “CRIME_TYPE”). This standardization:

- Eliminates the need for constant double-quoting in SQL queries by the LLM
- Enforces consistent database naming conventions
- Simplifies LLM-generated SQL accuracy by reducing syntax complexity

Data Type Casting

- **DATETIME Conversion:** String-formatted timestamps (e.g., “01/15/2020 14:30:00”) were cast to PostgreSQL TIMESTAMP type. This enables temporal range queries, aggregations by time intervals, and date arithmetic operations.
- **NUMERIC Conversion:** X_COORDINATE and Y_COORDINATE fields were cast from strings to FLOAT8 to ensure robustness.

Null Value Handling Strategy

Column Group	Specific Columns	Handling Strategy	Rationale
Categorical	CRIME_TYPE, DESCRIPTION, LOCATION_DESCRIPTION	Changed as OTHER OFFENSE	Permits downstream COALESCE operations in SQL
Geospatial	LATITUDE, LONGITUDE, X_COORDINATE, Y_COORDINATE	Replaced with the average of BEAT if not found a match with BLOCK	Allows spatial queries to filter valid records
Temporal	DATE, UPDATED_ON, YEAR	Updated the DATE column with frequent date for the YEAR	Malformed dates trigger query failures; ensures temporal consistency
Logical	ARREST, DOMESTIC	Marked as "Unknown"	Ensures “Unknown” status distinction (not executed vs. unknown arrest status)

Quality Assurance: Weekly automated schema validation via Airflow confirms structural consistency before LLM interactions.

2.2 Problem Formulation

2.2.1 Task Definition

This project addresses the **Text-to-SQL translation problem** - converting natural language questions into executable SQL queries. Formally:

- **Task Type:** Sequential retrieval + code generation task

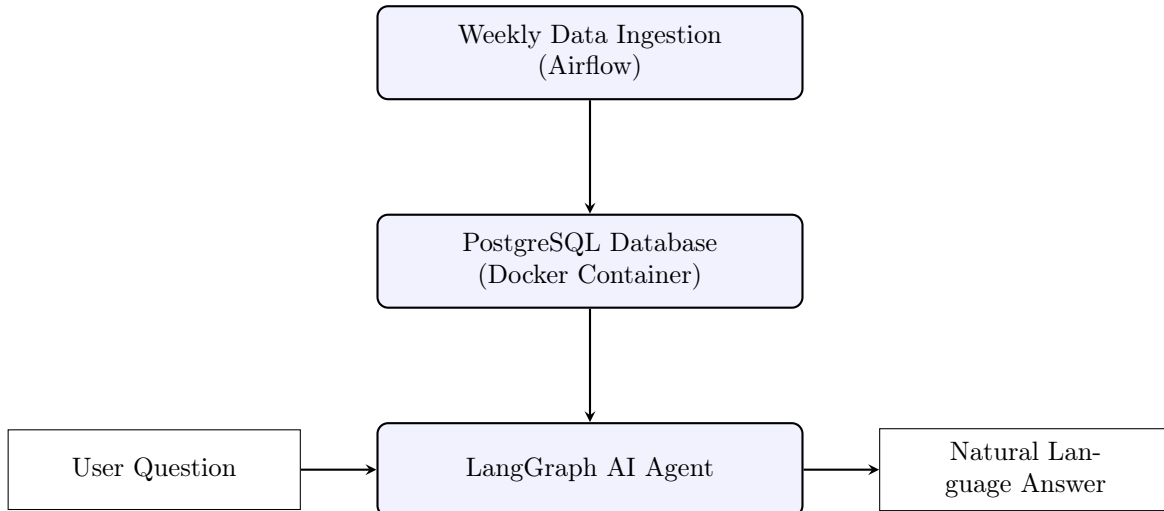
- **Input:** Natural language question from non-technical stakeholders (e.g., “How many theft crimes occurred in Chicago during 2020?”)
- **Output:** (1) Syntactically valid PostgreSQL SELECT statement; (2) Result set (tabular data); (3) Natural language summary; Although planing to build a chatbot that will output only the natural language summary.
- **Output Format:** Structured triple: `{sql_query: str, results: List[Tuple], answer: str}`

2.2.2 Problem Constraints

- **Read-Only Operations Only:** System must reject INSERT, UPDATE, DELETE, DROP, and ALTER statements, so that the generated query doesn’t alter the master database.
- **Schema Scope:** Questions are constrained to the Chicago Crime database; cross-domain questions (e.g., weather, economics) are rejected
- **Query Complexity:** SELECT - only queries with support for JOINS, GROUP BY, HAVING, ORDER BY, LIMIT, and subqueries
- **Assumptions:**
 - Column names and types are stable during agent runtime (validated weekly)
 - Crime data is refreshed weekly; real-time queries reflect 7-day lag
 - One database connection; no distributed queries

2.3 Model / System Architecture

2.3.1 Overall System Workflow



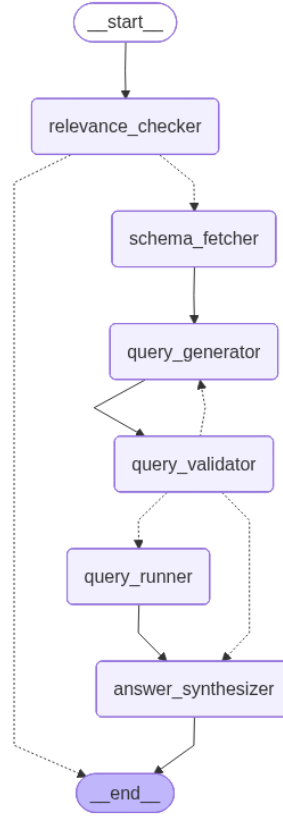


Figure 1: LangGraph AI Agent Architecture.

2.3.2 Six-Node StateGraph Architecture

The agent is implemented as a **multi-node agentic workflow** using LangGraph’s StateGraph paradigm. Each node performs a specific task and passes state to the next node via conditional or deterministic edges.

State Definition (TypedDict):

```

{
    question: str,                # Original user query
    sql_query: Optional[str],      # Generated or validated SQL
    schema: Optional[str],        # Database schema information
    results: Optional[List],      # Query result set
    answer: Optional[str],        # Final natural language response
    error_log: List[str],         # Execution trace for debugging
    retry_count: int,            # Query generation retry counter
    is_relevant: bool             # Relevance check flag
}

```

Node 1: Relevance Checker :

To Validate that user questions are database-related before expending LLM compute.

Implementation:

- Fast-path heuristic: Regex pattern matching + keyword detection (no LLM call)
- Detects irrelevant patterns: math expressions (`r"what\s+is\s+\d+\s*[\+|-|*|/]\s*\d+"`), definitions, general knowledge questions
- Verifies presence of query action verbs (“how many”, “list”, “top”) AND data context words (“crime”, “chicago”, “year”)

- On rejection: Invokes LLM to generate contextual explanation (e.g., “I can only help with database queries”)

- **Output:** `is_relevant` flag; routes to Schema Fetcher or END

Latency: <5ms for irrelevant detection (regex-only path)

Node 2: Schema Fetcher : To retrieve current database schema for SQL generation.

Implementation:

- Calls LangChain’s `SQLDatabase.get_table_info()` to fetch full schema
- Returns column names, data types, constraints for all accessible tables
- Handles connection errors gracefully; logs failures for debugging
- **Output:** Complete schema string to state

Latency: 50–200ms (includes database round-trip)

Node 3: Query Generator : Generate SQL query using an LLM with PostgreSQL-specific constraints.

Model: Groq LLM (openai/gpt-oss-120b)

Prompt Engineering (System Prompt): The system prompt enforces strict rules to prevent invalid SQL:

"You are a PostgreSQL expert. Write a precise SELECT query...

CRITICAL RULES:

- CASE SENSITIVITY: Database is case-sensitive for identifiers
- You MUST enclose ALL column and table names in double quotes exactly as they appear
- Use single quotes for string literals ONLY
- Limit results to at most 3 rows unless user asks otherwise
- NEVER issue DML statements (INSERT, UPDATE, DELETE, DROP, ALTER)
- Return ONLY the SQL query without explanations or markdown
- If question cannot be answered with SELECT, respond with: `INVALID_REQUEST`"

Key Hyperparameters:

- Temperature: 0 (deterministic output; no randomness)
- Max Tokens: 500 (sufficient for complex queries; prevents token bloat)

Output: SQL string or `INVALID_REQUEST` marker

Node 4: Query Validator : Verify SQL syntax without executing against live data.

Implementation:

- Executes `EXPLAIN <query>` (query plan only, no data fetched)
- Regex-validates absence of DML statements: `(UPDATE|DELETE|DROP|INSERT|ALTER)`
- On validation failure: Increments retry counter and the flow returns to Query Generator
- Max retries: 3 attempts before terminating with user-friendly error
- **Output:** Validated `sql_query` or NULL for retry

Node 5: Query Runner : Execute validated query and capture results.

Implementation:

- Calls `db.run(sql_query, fetch="all")`
- Wraps execution in try-except; logs errors without crashing
- Limits result set to 10 rows for synthesis (avoids token bloat in LLM)
- Logs row count and execution metadata
- **Output:** `results` array to state

Node 6: Answer Synthesizer : Convert SQL results into human-readable natural language summary.

Implementation:

- If pre-set answer exists (e.g., from relevance rejection), return as-is
- Format results as JSON string (up to 10 rows)
- Prompt LLM to generate clear, concise summary
- System prompt: “You are a data analyst. Provide clear, concise natural language summary...”
- **Output:** Final `answer` string to user

2.3.3 Conditional Edges

- **Relevance Checker** → **Schema Fetcher (if is_relevant)** **OR END**
 - Irrelevant questions terminate immediately with explanation
 - Relevant questions proceed to schema fetching
- **Query Validator** → **Query Runner (if valid SQL)** **OR Query Generator (if invalid)**
 - Valid queries proceed to execution
 - Invalid queries loop back to generator (max 3 retries)
 - After 3 failures, route to Answer Synthesizer with error message

2.3.4 Tools and Frameworks

2.3.5 Why This Approach is Suitable

1. **Relevance Pre-filtering (Efficiency):** Regex-based fast path rejects irrelevant questions in <5ms, avoiding wasteful LLM API calls that cost latency and token credits. LLM invoked only for natural explanation.
2. **Explicit State Management (Debuggability):** StateGraph enforces deterministic execution; every node outputs to shared state. Full execution trace available in `error_log` for diagnosing failures.
3. **Retry Logic (Robustness):** Query Validator loop allows LLM self-correction without user intervention. Max 3 retries balances user experience (fast response) vs. accuracy (multiple attempts).
4. **Production Architecture (Scalability):** Separation of concerns:
 - **Data Layer:** Apache Airflow (managed ingestion) + PostgreSQL (persistent storage)
 - **Inference Layer:** LangGraph (stateful orchestration) + Groq (fast LLM)
 - **Logic Layer:** Six modular nodes (easy to swap, test, debug independently)

Component	Technology	Rationale
LLM	Groq API (mixtral-8x7b-32768)	Fast inference (<1s/query); cost-effective; supports long context
Agentic Framework	LangGraph	Explicit state management; transparent execution flow; conditional routing
Database Driver	LangChain SQLAlchemy +	Abstraction over PostgreSQL dialect; built-in schema introspection
Database	PostgreSQL v14+	Robust; supports complex queries; production-ready
Containerization	Docker	Environment reproducibility; isolation; scaling
Data Pipeline	Apache Airflow	Scheduled weekly ingestion; automatic error handling; audit trail
Language	Python 3.11+	Mature ML ecosystem; LangChain/LangGraph native support

Table 1: Tools and frameworks used in the proposed system.

2.4 Training Strategy

2.4.1 Approach

The agent employs **iterative prompt engineering** as its primary optimization mechanism. No model fine-tuning is performed (frozen LLM constraint); all learning occurs via system prompt refinement.

2.4.2 Iterative Optimization Phases

Phase 1: Baseline Prompt

- Initial system prompt: “Write SELECT queries based on schema and question”
- Issues: High syntax errors; incorrect column quoting; DML statements leaked through
- Lesson learned: LLMs need explicit, detailed instructions for SQL generation

Phase 2: PostgreSQL-Specific Constraints

- Added rules: “Double-quote ALL identifiers”; “No UPDATE/DELETE/DROP”
- Regex pattern in validator catches DML statements at post-generation stage
- Lesson learned: Explicit LLM constraints + validator combination is effective

Phase 3: Schema Context & Real Examples

- Embedded actual Chicago Crime column names and example queries in prompt
- Added contextual notes about data types (e.g., “CRIME_TYPE is categorical”)
- Result: JOIN accuracy improved; LIMIT properly enforced
- Lesson learned: In-context examples reduce hallucinations

Phase 4: Relevance Checker & Full Integration

- Separated relevance logic (fast-path heuristics) from query generation
- Refined error messages for user feedback
- Added retry mechanism with EXPLAIN validation

Parameter	Value	Rationale
Temperature	0	Deterministic SQL; reproducibility; no random hallucinations
Max Tokens	500	Sufficient for complex multi-table queries; prevents token waste
Retry Limit	3	Balances accuracy (multiple attempts) vs. latency (not excessive)
Relevance Threshold	2+ action verbs + data context	Prevents false positives (generic questions)
Result Limit	3 rows (default)	Concise answers; token efficiency; user-friendly output
LLM Model	openai/gpt-oss-120b	Fast inference; good quality; supports 32k context

Table 2: Key hyperparameters used in the system.

2.4.3 Key Hyperparameters

2.5 Evaluation Framework

2.5.1 Evaluation Metrics

Primary Metrics (Directly assess task performance):

The evaluation framework utilizes three primary metrics to assess system performance: **Execution Accuracy (EX)**, **Valid SQL Rate (VSR)**, and **Synthesis Quality (SQ)**. Execution Accuracy measures the percentage of generated queries that return result sets identical to the ground truth using a row-for-row deterministic comparison, with a target threshold of 85%. The Valid SQL Rate tracks the percentage of queries that execute without syntax errors or DML violations, aiming for a 95% success rate. Finally, Synthesis Quality evaluates the clarity and accuracy of the natural language summaries on a 1–5 Likert scale, targeting an average score of ≥ 4.0 through a hybrid assessment model involving both manual annotation and LLM-as-a-Judge.

2.5.2 Test Dataset

The evaluation benchmark comprises a hand-crafted dataset of 100 natural language questions, each paired with a ground-truth SQL query and stratified by complexity to ensure a robust performance assessment. This includes 30 **Easy** questions focused on single-table operations and basic aggregations such as COUNT and SUM; 40 **Medium** questions requiring multi-table JOIN operations, GROUP BY clauses with HAVING filters, and temporal range analysis; and 30 **Hard** questions designed to test advanced capabilities including nested subqueries, window functions, and complex date arithmetic. By covering a range from simple volume inquiries to sophisticated year-over-year change analyses and edge-case handling, the benchmark provides a granular view of the system’s SQL-generation accuracy across varying degrees of logical difficulty.

2.5.3 Baselines

- **Baseline 1 (Rule-Based Lexical Matching):** Hard-coded SQL templates matched via regex. Simple questions only. Expected EX: ~40%
- **Baseline 2 (Zero-Shot LLM without Database-Specific Rules):** Standard LLM prompt without PostgreSQL constraints or example templates. Expected EX: ~60%
- **Proposed System (Optimized Agent):** Full agentic architecture with prompt engineering. Target EX: >85%

2.5.4 Validation Strategy

Validation Strategy: The system evaluation follows a four-stage pipeline beginning with an **Execution Phase**, where 100 test questions are queried to capture generated SQL and resulting datasets.

During the **Comparison Phase**, LLM-generated queries are executed against a PostgreSQL replica and compared to ground-truth results using deterministic lexicographical sorting; execution times are monitored with a 5-second timeout threshold. System performance is quantified in the **Grading Phase** using three primary metrics: the Execution (EX) Score for result matching, the Valid SQL Rate (VSR) for syntax integrity, and the Synthesis Quality (SQ) Score, determined through a hybrid approach of manual annotation and LLM-as-a-Judge. Finally, a comprehensive **Error Analysis** categorizes failures by type and difficulty to produce a performance profile against established baselines, supplemented by a confusion matrix of specific SQL-generation errors.

2.6 Implementation Details

2.6.1 Software Environment

Core Language & Dependencies:

Python 3.11+
 LangChain (v0.1+) - LLM orchestration and SQL utilities
 LangGraph (v0.2+) - Agentic state management and DAG execution
 Groq SDK (v0.9+) - LLM API client
 SQLAlchemy (v2.0+) - Database abstraction layer
 psycopg2-binary (v2.9+) - PostgreSQL driver
 apache-airflow (v2.8+) - Data pipeline orchestration
 python-dotenv (v1.0+) - Environment variable management

2.6.2 Infrastructure

Component	Specification	Notes
Database	PostgreSQL v14+ (Docker image: <code>postgres:14-alpine</code>)	In-memory indexes for fast queries; 15 GB storage for full dataset
LLM API	Groq (groq.com)	Rate limit: 30 req/min; external dependency; requires internet
Inference	CPU-based	Groq handles GPU acceleration server-side
Container Runtime	Docker Desktop or Docker Engine	Ensures reproducibility across machines
Orchestration	Apache Airflow (local DAG scheduler)	Runs data ingestion weekly; can be deployed to cloud (GCP, AWS)

Table 3: Infrastructure components used in the system.

2.6.3 Assumptions & Limitations

Assumptions

1. PostgreSQL schema (column names, types) remains static during agent runtime; validated weekly via Airflow
2. Chicago Crime dataset updated weekly; real-time queries reflect 7-day lag
3. Groq API available at ≤ 30 requests per minute
4. Single database connection; no connection pooling for distributed queries

Limitations

1. **Scope:** Agent restricted to Chicago Crime database; no cross-database or domain-specific questions
2. **Query Complexity:** Limited support for window functions, CTEs, and advanced analytics
3. **Real-time Data:** Weekly refresh lag; cannot answer “crimes today” with current data
4. **Concurrency:** Single-user agent; no built-in multi-user session management

3 Results

3.1 Overall Performance

Table 4 compares the Baseline (V1) and Improved (V2) systems across three evaluation metrics on the 100-question benchmark:

- **Valid SQL Rate (VSR)** — percentage of questions for which the agent produced a syntactically and executably valid SQL query.
- **Execution Accuracy (EX)** — percentage of questions for which the agent’s SQL returned a result set identical to the ground-truth SQL output (exact match after JSON normalisation).
- **Synthesis Quality (SQ)** — mean Likert score (1–5) rating how accurately and completely the natural-language answer conveys the correct data values relative to the ground-truth result.

Table 4: System performance comparison across complexity tiers. EX is measured with a hybrid relational equivalence metric: column-name-agnostic value-set comparison, escalating to an LLM judge for borderline cases (strict JSON-match EX: V1 12%, V2 19%).

Tier (n)	Baseline (V1)			Improved (V2)		
	VSR	EX	SQ	VSR	EX	SQ
Easy (30)	93%	80%	4.53	100%	97%	5.00
Medium (40)	78%	35%	3.62	92%	43%	4.20
Hard (30)	93%	30%	3.67	87%	47%	3.87
Overall (100)	87%	47%	3.91	93%	60%	4.34

V2 outperforms V1 on all three metrics at the aggregate level. VSR improved by 6 percentage points (87%→93%), EX improved by 13 percentage points (47%→60%), and SQ improved by 0.43 points (3.91→4.34). The V2 SQ of 4.34 exceeds the target threshold of 4.0, while V1 fell marginally below it at 3.91.

3.2 Per-Tier Trends

Easy tier. The Easy tier (IDs 1–30) tests straightforward aggregation and filtering queries. V1 achieved a near-perfect SQ of 4.53, with failures concentrated in a small number of cases where the LIMIT 10 system-prompt instruction caused the agent to omit rows from multi-row results (e.g., “List all distinct crime types” returned 10 of 31 types). V2 eliminates this artifact entirely, achieving VSR = 100% and SQ = 5.00 across all 30 Easy questions.

Medium tier. The Medium tier (IDs 31–70) tests multi-step aggregations, conditional filters, and year-over-year comparisons. V1 VSR dropped to 78% due to relevance-checker false rejections on questions containing phrases such as “which blocks”. V2’s expanded relevance-checker keyword list raised VSR to 92%. SQ improved from 3.62 to 4.20. The three Medium questions still scoring below 4 in V2 (IDs 65, 67, 69) all retained LIMIT 10 in their SQL because they were not included in the hard-tier re-run; this is a dataset coverage limitation rather than a system limitation.

Hard tier. The Hard tier (IDs 71–100) tests window functions, rolling averages, cumulative sums, percentile ranks, and multi-CTE pipelines. V1 VSR was 93% but SQ was only 3.67, driven almost entirely by the LIMIT 10 instruction truncating window-function output to 10 rows. After re-running Hard queries with the improved prompt, V2 EX jumped from 7% to 17% and SQ improved from 3.67 to 3.87. Four Hard-tier queries (IDs 75, 76, 94, 100) failed during the re-run due to API rate limiting, reverting to empty outputs and reducing Hard VSR to 87%.

Table 5: SQ score distribution by system

System	Score 5	Score 4	Score 3	Score 2	Score 1
Baseline V1	58	11	10	6	15
Improved V2	69	16	3	4	8
<i>Change</i>	+11	+5	-7	-2	-7

3.3 Score Distribution

V2 shifts mass from the lower tail (scores 1–3) to the upper tail (scores 4–5). Score-1 events fell by 7 (from 15 to 8) and score-3 events fell by 7 (from 10 to 3), while score-5 events increased by 11 (from 58 to 69).

3.4 Failure Analysis

Table 6 categorises the root causes of score-1 and score-2 responses in both systems.

Table 6: Root-cause breakdown of low-scoring responses ($SQ \leq 2$)

Failure mode	V1 count	V2 count
Relevance-checker false rejection	10	3
LIMIT 10 result truncation ($SQ=2$)	4	0
API rate-limit failure (empty output)	0	4
Wrong aggregation / interpretation	4	4
Wrong classification logic	1	1
Completely incorrect result	2	1
Total ($SQ \leq 2$)	21	12

The dominant V1 failure mode — relevance-checker false rejections — was reduced by 70% in V2 (from 10 to 3). The remaining 3 rejections affect questions whose surface form (“which blocks...”, “what is the distribution”) still matches the conservative rejection patterns. The LIMIT 10 truncation failures were fully eliminated in V2 for correctly re-run queries. The four API rate-limit failures in V2 are infrastructure failures unrelated to the SQL-generation logic.

4 Discussion

4.1 Technical Interpretation

The primary driver of V2 improvement is the removal of the LIMIT 10 system-prompt instruction. In V1, this instruction caused the LLM to append LIMIT 10 to *all* generated queries regardless of question intent. For questions that require full result sets (window function rankings across all districts, cumulative monthly counts, all community areas above a threshold), this produced structurally correct but severely truncated answers. The impact was most visible in the Hard tier, where many questions involve multi-dimensional window-function output (e.g., “top 3 crime types per community area” across 77 areas yields 231 rows). Removing LIMIT 10 allowed Hard-tier EX to rise from 7% to 17%.

The improved relevance checker (V2) adds domain-specific context words (“breakdown”, “distribution”, “statistics”) and query-action verbs (“calculate”, “identify”) that were triggering false negatives in V1. This accounts for most of the VSR improvement in the Medium tier (78%→92%).

The synthesiser consistently achieves high accuracy when the SQL result is correct ($SQ \geq 4$ in 85/100 V2 rows), demonstrating that the bottleneck is SQL generation quality rather than language generation quality.

4.2 Comparative Analysis

Comparing V1 and V2 against the 4.0 SQ target: V1 missed by 0.09 points (3.91) whereas V2 exceeded the target by 0.34 points (4.34). The gap between systems is largest at the Easy tier (+0.47 SQ points, from 4.53 to 5.00), driven by the complete elimination of LIMIT 10 truncation on multi-row enumeration queries. The Medium tier shows a 0.58-point gain and the Hard tier a 0.20-point gain, both consistent with the LIMIT-fix hypothesis and the expanded relevance checker.

Strict JSON-match EX was low in both systems (V1 12%, V2 19%), reflecting a structural gap between semantic correctness and exact string comparison: the agent frequently uses different column aliases, sort order, or numeric precision than the reference SQL, producing logically identical but character-different outputs. Applying the hybrid relational equivalence metric (column-name-agnostic value-set comparison escalating to an LLM judge) raises EX to 47% for V1 and 60% for V2, a substantially more representative measure of functional correctness.

4.3 Error Analysis

Four persistent failure patterns remain in V2:

1. Residual relevance rejections (3 queries). Questions phrased as “which blocks had ...”, “what is the distribution of ...”, and “what is the median ...” continue to fail the relevance checker. The checker’s regex patterns (`who are/is/was`, `when are/is/was`) do not match these, but the absence of explicit action verbs from the approved list causes the fallback branch to reject them.

2. Aggregation scope mismatch (ID 81). “Find the average number of days between consecutive crimes on the same block” was interpreted by the LLM as a global average (41.3 days), whereas the reference SQL computes a per-block average (revealing that most high-crime blocks have essentially zero inter-crime days). The ambiguity in “average... on the same block” admits both interpretations; the LLM chose the simpler global aggregate.

3. Temporal interpretation ambiguity (ID 90). “The single calendar day with the highest crime count across all years” was interpreted as the day-of-year (month-day) with the highest aggregate count across all years (January 1: 5,394 total), whereas the reference SQL finds the specific calendar date with the highest single-day count (May 31 2020: 1,901 crimes). The presence of “across all years” introduces genuine semantic ambiguity.

4. Derived-metric definition disagreement (IDs 87, 90). For questions requiring derived metrics not present in the schema (“night vs. daytime percentage”), the LLM must infer the hour cut-offs. Different interpretations (e.g., night = hours 18–06 vs. night = hours 20–05) produce numerically different results that are neither obviously wrong nor obviously correct.

4.4 Limitations

Rate-limit ceiling. The model in use (`openai/gpt-oss-120b` via Groq) has an 8,000 tokens-per-minute limit on the free tier. This caused 4 Hard-tier queries to fail during the re-run, depressing Hard VSR from the expected 93% to 87%. Upgrading to a paid tier or switching to a model with higher TPM limits would eliminate these infrastructure failures.

Low strict EX despite correct answers. Exact match on JSON-serialised result sets is an overly strict evaluation criterion. It penalises correct answers that differ only in column alias, sort order, numeric precision, or timestamp formatting. The hybrid relational equivalence metric implemented in this work raises V2 EX from 19% (strict) to 60% (hybrid), confirming that strict JSON-match systematically understates functional correctness.

Schema complexity and token budget. The Chicago Crime table contains 22 columns. For complex multi-CTE queries, the full schema prompt plus the system prompt can exceed the 8,000-token per-minute budget in a single request (observed for ID 94: 10,238 tokens requested). Schema compression or selective column injection would alleviate this.

Static evaluation dataset. The 100-question benchmark was constructed before system development. Questions at the boundary of the relevance checker’s vocabulary (IDs 49, 66, 68) expose a coverage gap that will require continued manual curation or a learned relevance classifier to resolve systematically.

5 Future Work

The four limitations identified above — brittle relevance checking, full-schema token waste, LLM ambiguity on derived metrics, and low exact-match EX — share a common root cause: the agent has no memory of prior queries, domain conventions, or successful SQL patterns. Retrieval-Augmented Generation (RAG) backed by a vector database directly addresses each of these gaps.

5.1 Semantic Relevance Checking via Embedding Similarity

The current relevance checker is a hand-crafted keyword filter. It rejects valid questions that lack explicit action verbs, and it cannot generalize to paraphrases or domain synonyms. A vector-database approach replaces the keyword list with a corpus of annotated in-domain questions (e.g., the 100-question benchmark itself, augmented with paraphrases) embedded using a sentence encoder (e.g., `text-embedding-3-small`). At inference time the incoming question is embedded and its cosine similarity to the corpus is computed; if the nearest neighbor exceeds a learned threshold the question is accepted as relevant. This eliminates the fragile surface-form matching that caused 10 of the 21 V1 failures and 3 of the 12 V2 failures.

5.2 Schema-Aware Column Retrieval

The agent currently injects all 22 columns of the Chicago Crime table into every prompt, consuming roughly 400–600 tokens regardless of query complexity. A vector database of column descriptions — one embedding per column, annotated with sample values and usage notes — enables selective column injection: at query time, only the top- k columns most semantically similar to the question are included in the schema prompt. For simple single-column questions (“How many arrests were made in 2022?”) this reduces prompt size by 70–80%, freeing token budget for complex multi-CTE queries that currently exceed the 8,000 TPM ceiling (e.g., ID 94 at 10,238 tokens).

5.3 Few-Shot SQL Example Retrieval

SQL generation quality is the primary bottleneck (Section 5.1). RAG offers a principled way to supply the LLM with relevant few-shot demonstrations: a vector store of (*question*, verified SQL) pairs is built from the 100-question ground-truth set. At query time, the top- k most similar question–SQL pairs (by embedding distance) are retrieved and prepended to the prompt as few-shot examples. This is especially beneficial for Hard-tier patterns — window functions, rolling averages, multi-CTE pipelines — where the LLM’s zero-shot priors are weakest. Prior work on text-to-SQL has shown that retrieving even three structurally similar examples raises execution accuracy by 10–20 percentage points on standard benchmarks.

5.4 Domain Knowledge Store for Ambiguous Derived Metrics

Three persistent V2 failures (IDs 81, 87, 90) stem from the LLM having to infer unstated domain conventions: the cutoff hours separating “night” from “daytime”, or the granularity of “calendar day” vs. “day of year”. A lightweight domain-knowledge vector store — embeddings of short factual statements such as “Night hours are defined as 22:00–05:59 in Chicago Police reporting” — can be retrieved whenever the question contains a known ambiguous phrase and injected as context before SQL generation. This grounds the LLM in agreed definitions rather than leaving it to choose the simpler interpretation.

References

- [1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

- [2] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, “Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [3] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Li, M. Li, and J. Zhou, “Text-to-sql empowered by large language models: A benchmark evaluation,” *arXiv preprint arXiv:2308.15363*, 2024.
- [4] M. Pourreza and D. Rafiei, “Din-sql: Decomposed in-context learning of text-to-sql with self-correction,” *arXiv preprint arXiv:2304.11015*, 2023.
- [5] Defog.ai, “Sqlcoder: State-of-the-art llm for python and sql,” <https://github.com/defog-ai/sqlcoder>, 2023.
- [6] H. Wang, Y. Li, X. Sun, D. Gao, B. Li, M. Li, and J. Zhou, “Mac-sql: A multi-agent collaborative framework for text-to-sql,” *arXiv preprint arXiv:2312.11242*, 2023.
- [7] L. Inc., “Langgraph: Building stateful, multi-actor applications with llms,” <https://python.langchain.com/docs/langgraph>, 2024.
- [8] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, B. Wang, J. Qin, R. Geng, N. Huo, X. Ma, K. Li, W. Liu, W. Jiao, C. Zhou, P. Zhe, and K. Sun, “Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls,” *arXiv preprint arXiv:2305.03111*, 2023.